PRAKTIKUM 3

Eksplorasi HDFS Block, Replication, dan Data Locality

Mata Kuliah: Big Data
Hadoop 3.x – HDFS & MapReduce

1. Deskripsi Praktikum

Praktikum ini merupakan kelanjutan dari Praktikum 1 (HDFS) dan Praktikum 2 (MapReduce WordCount). Fokusnya adalah melihat secara langsung bagaimana file di HDFS dipecah menjadi block, bagaimana block memiliki replica, di DataNode mana block tersebut disimpan, serta bagaimana konsep data locality berkaitan dengan pemrosesan MapReduce.

Pada praktikum sebelumnya mahasiswa sudah melakukan kompilasi, packaging JAR, submit job MapReduce, melihat output, dan melihat YARN UI. Karena itu, praktikum ini tidak mengulang proses tersebut dari awal, tetapi memperdalam hubungan antara HDFS, DataNode, YARN, dan MapReduce.

2. Tujuan Praktikum

- Memahami hubungan antara file, block, replica, dan DataNode.
- Mengamati lokasi block dan replica menggunakan HDFS fsck.
- Memahami fungsi replication factor untuk fault tolerance.
- Membandingkan file kecil dan file yang lebih besar dari sisi jumlah block.
- Memahami konsep data locality dalam pemrosesan MapReduce.
- Menghubungkan penyimpanan HDFS dengan eksekusi MapReduce melalui YARN.

3. Prasyarat

- Docker Desktop sudah terpasang dan Docker Engine sedang berjalan.
- Cluster Hadoop dari Praktikum 2 dapat dijalankan.
- Cluster terdiri dari 1 NameNode, 2 DataNode, 1 ResourceManager, dan 2 NodeManager.
- File WordCount JAR dari Praktikum 2 sudah tersedia di container NameNode, misalnya /tmp/wordcount.jar.
- Seluruh perintah Windows pada modul ini menggunakan Command Prompt (CMD), bukan PowerShell.

4. Aplikasi yang Digunakan

Aplikasi	Digunakan untuk	Lingkungan
Docker Desktop	Menjalankan container Hadoop	Windows
Command Prompt (CMD)	Menjalankan docker dan perintah Windows	Windows

Terminal Linux dalam container	Menjalankan perintah Linux/Hadoop di container	Container
Browser	Melihat HDFS UI dan YARN UI	Windows

Catatan: OS Mac harap menyesuaikan

5. Arsitektur Cluster

Gunakan arsitektur yang sama dengan Praktikum 2:

- NameNode → mengelola metadata HDFS dan informasi lokasi block.
- DataNode 1 dan DataNode 2 → menyimpan block data HDFS.
- ResourceManager → mengelola resource dan penjadwalan YARN.
- NodeManager 1 dan NodeManager 2 → mengelola container/task pada masing-masing worker.

Gambaran alur:

```
File → HDFS → Block → Replica → DataNode

MapReduce Job → YARN ResourceManager → NodeManager → Mapper/Reducer

Input HDFS → Map → Shuffle & Sort → Reduce → Output HDFS
```

6. Verifikasi Container Hadoop

6.1 Buka Command Prompt (CMD)

Tekan Windows + R, ketik cmd, lalu Enter.

```
cd C:\bigdata\mapreduce

docker ps
```

Catatan: Perintah docker ps dijalankan di CMD Windows.

Pastikan container Hadoop yang dibutuhkan sedang berjalan, termasuk NameNode, DataNode, ResourceManager, dan NodeManager.

6.2 Periksa DataNode melalui HDFS

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfsadmin -report
```

Perhatikan bagian Live datanodes. Untuk praktikum ini diharapkan terdapat 2 DataNode aktif.

7. Memeriksa Replication Factor

```
docker exec -it namenode /opt/hadoop/bin/hdfs getconf -confKey dfs.replication
```

Catat nilai replication factor default. Jika nilainya 1, file yang dibuat dengan konfigurasi default hanya memiliki satu replica.

Catatan: Replication factor menunjukkan jumlah salinan block yang ingin dipertahankan HDFS. Replica bukan berarti data diproses berulang-ulang. Replica digunakan terutama untuk ketahanan terhadap kegagalan node.

8. Membuat File Kecil di CMD

Buat folder kerja di Windows:

```
cd C:\bigdata\mapreduce

mkdir praktikum3

cd praktikum3
```

Buat file input.txt menggunakan CMD:

```
echo Hadoop HDFS DataNode MapReduce Big Data>input.txt

echo Hadoop stores data as blocks and replicas>>input.txt

echo MapReduce processes data stored in HDFS>>input.txt
```

Periksa ukuran file menggunakan CMD:

```
dir input.txt
```

Catatan: Perintah dir adalah perintah CMD. Jangan menggunakan Get-Item karena itu adalah sintaks PowerShell.

9. Menyalin File ke Container NameNode

```
docker cp input.txt namenode:/tmp/input.txt
```

Verifikasi file di container:

```
docker exec -it namenode ls -lh /tmp/input.txt
```

10. Upload input.txt ke HDFS dengan Replication Factor 2

Buat folder HDFS:

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /praktikum/hdfs/input
```

Upload file dan tetapkan replication factor 2:

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -Ddfs.replication=2 -put -f /tmp/input.txt
/praktikum/hdfs/input/
```

Periksa file di HDFS:

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/hdfs/input
```

Perhatikan kolom replication. File diharapkan memiliki replication factor 2.

11. Melihat Block dan Lokasi Replica

Jalankan HDFS fsck:

```
docker exec -it namenode /opt/hadoop/bin/hdfs fsck /praktikum/hdfs/input/input.txt -files -
blocks -locations
```

Perhatikan informasi:

- Block ID, misalnya blk_....
- Jumlah block.
- Jumlah replica.
- Nama DataNode tempat replica berada.

Catatan: Karena input.txt berukuran kecil dan lebih kecil dari ukuran block, file biasanya hanya membutuhkan satu block. Dengan replication factor 2, block tersebut memiliki dua replica yang dapat berada pada dua DataNode.

12. Membuat File Besar ±5 MB Menggunakan CMD

Pastikan berada di folder praktikum3 pada CMD:

```
cd C:\bigdata\mapreduce\praktikum3
```

Buat file besar berisi teks menggunakan perulangan CMD berikut:

```
for /L %i in (1,1,125000) do @echo Hadoop HDFS DataNode MapReduce Big Data>>bigfile.txt
```

Perintah tersebut membuat 125.000 baris teks. Ukuran akhirnya mendekati 5 MB.

Periksa ukuran file:

```
dir bigfile.txt
```

Catatan: Jika ukuran tidak persis 5 MB, tidak masalah. Yang penting file cukup besar untuk melewati ukuran block yang digunakan pada percobaan berikutnya.

Catatan: Perintah for /L di atas ditujukan untuk diketik langsung pada CMD. Jika ditulis ke file .bat, %i harus ditulis sebagai %%i.

13. Menyalin bigfile.txt ke NameNode

```
docker cp bigfile.txt namenode:/tmp/bigfile.txt

docker exec -it namenode ls -lh /tmp/bigfile.txt
```

14. Memeriksa Ukuran Block HDFS

```
docker exec -it namenode /opt/hadoop/bin/hdfs getconf -confKey dfs.blocksize
```

Perintah tersebut menampilkan block size default cluster. Nilai umum dapat berupa 134217728 byte (128 MB), tetapi ikuti nilai yang muncul pada cluster Anda.

Catatan: Jika block size default jauh lebih besar daripada bigfile.txt, bigfile.txt tetap hanya menjadi satu block. Karena itu pada percobaan berikut block size dibuat 1 MB agar mahasiswa dapat melihat pemecahan file menjadi beberapa block.

15. Upload bigfile dengan Block Size 1 MB dan Replication 2

Buat folder HDFS baru:

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /praktikum/hdfs/biginput
```

Upload bigfile dengan block size 1 MB dan replication factor 2:

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -Ddfs.blocksize=1048576 -Ddfs.replication=2 -
put -f /tmp/bigfile.txt /praktikum/hdfs/biginput/
```

Periksa file:

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/hdfs/biginput
```

16. Mengamati Block dan Replica bigfile.txt

```
docker exec -it namenode /opt/hadoop/bin/hdfs fsck /praktikum/hdfs/biginput/bigfile.txt -files -
blocks -locations
```

Catat hasil pengamatan:

- Ukuran file.
- Jumlah block.
- Block ID yang terbentuk.
- Jumlah replica setiap block.
- DataNode tempat masing-masing replica berada.

Dengan file sekitar 5 MB dan block size 1 MB, secara konsep akan terbentuk sekitar 5 block. Setiap block ditargetkan memiliki 2 replica.

17. Membandingkan input.txt dan bigfile.txt

File	Ukuran	Perkiraan jumlah block	Replica
input.txt	Kecil	Biasanya 1	2
bigfile.txt	±5 MB	Sekitar 5 jika block size 1 MB	2 per block

Gunakan hasil fsck Anda sebagai data final, bukan hanya perkiraan pada tabel.

18. Memproses bigfile dengan MapReduce WordCount

Tahap ini menghubungkan kembali HDFS dengan MapReduce yang sudah dipelajari pada Praktikum 2.

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -rm -r -f /praktikum/hdfs/bigoutput

docker exec -it namenode /opt/hadoop/bin/hadoop jar /tmp/wordcount.jar WordCount
/praktikum/hdfs/biginput /praktikum/hdfs/bigoutput
```

Periksa output:

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/hdfs/bigoutput

docker exec -it namenode /opt/hadoop/bin/hdfs dfs -cat /praktikum/hdfs/bigoutput/part-r-00000
```

Catatan: Jika nama file output berbeda, gunakan hasil hdfs dfs -ls untuk menentukan nama file yang benar.

19. Melihat Job pada YARN

Buka browser Windows dan akses:

```
http://localhost:8088
```

Amati:

- Application ID.
- Application Type.
- Final Status.
- Start Time dan Finish Time.
- Jumlah resource/task yang digunakan.

Hubungkan hasil ini dengan konsep ResourceManager dan NodeManager dari Praktikum 2.

20. Memahami Data Locality

Data locality adalah prinsip Hadoop untuk menjalankan task sedekat mungkin dengan lokasi data input. Jika block input tersedia pada DataNode tertentu, scheduler berusaha menempatkan task pada node tersebut atau sedekat mungkin dengan node tersebut sehingga perpindahan data melalui jaringan dapat dikurangi.

Alur konseptual:

```
HDFS Block berada di DataNode 1

        ↓

YARN menyiapkan container pada NodeManager

        ↓

Mapper membaca block

        ↓

Data diproses sedekat mungkin dengan sumbernya
```

Perlu dibedakan: DataNode menyimpan block; NodeManager mengelola container/resource di worker; Mapper/Reducer adalah task yang melakukan pemrosesan.

Jika satu block memiliki 2 replica, Hadoop tidak menjalankan Mapper dua kali hanya karena ada dua salinan. Satu task biasanya membaca salah satu replica yang tersedia. Replica lainnya merupakan salinan untuk fault tolerance dan dapat digunakan apabila replica yang sedang diakses tidak tersedia.

Jika file terdiri dari beberapa block, beberapa Mapper dapat bekerja secara paralel pada input split yang berbeda. Hal tersebut berbeda dari menjalankan pemrosesan terhadap seluruh replica.

21. Melihat Output HDFS

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum/hdfs/bigoutput

docker exec -it namenode /opt/hadoop/bin/hdfs fsck /praktikum/hdfs/bigoutput -files -blocks -
locations
```

Amati kembali hubungan antara output MapReduce dan penyimpanan HDFS.

22. Tugas Praktikum

1. Buktikan bahwa input.txt tersimpan sebagai block HDFS dan memiliki 2 replica.
2. Tentukan DataNode yang menyimpan setiap replica input.txt.
3. Buat bigfile.txt menggunakan CMD dan catat ukuran aktualnya dari perintah dir.
4. Tentukan jumlah block bigfile.txt setelah menggunakan block size 1 MB.
5. Catat DataNode yang menyimpan setiap replica block bigfile.txt.
6. Jalankan WordCount pada bigfile.txt dan dokumentasikan Application ID pada YARN.
7. Jelaskan hubungan DataNode, NodeManager, Mapper, Reducer, dan ResourceManager pada percobaan.
8. Jelaskan mengapa replica tidak berarti data diproses dua kali.

23. Daftar Screenshot Laporan

- Output docker ps.
- Output hdfs dfsadmin -report yang menunjukkan DataNode aktif.
- Output replication factor.
- Output dir input.txt.
- Output hdfs dfs -ls untuk input.txt.
- Output hdfs fsck input.txt -files -blocks -locations.
- Output dir bigfile.txt.
- Output hdfs getconf -confKey dfs.blocksize.
- Output hdfs fsck bigfile.txt -files -blocks -locations.
- YARN UI yang menunjukkan job WordCount.
- Output WordCount pada HDFS.

24. Troubleshooting

Masalah	Tindakan
docker ps kosong	Pastikan Docker Desktop sudah berjalan dan jalankan kembali docker compose up -d dari folder project Hadoop.
DataNode tidak muncul pada dfsadmin -report	Periksa docker ps dan log container DataNode dengan docker logs datanode1 atau nama container sesuai compose Anda.
File tidak ditemukan saat docker cp	Pastikan CMD berada di folder yang berisi file dan gunakan dir untuk memeriksa nama file.
File sudah ada di HDFS	Gunakan -f pada perintah put atau hapus file/folder output terlebih dahulu.
WordCount gagal karena output sudah ada	Hapus output menggunakan hdfs dfs -rm -r -f sebelum menjalankan job lagi.
Tidak terlihat beberapa block	Periksa kembali block size. Jika block size terlalu besar dibanding ukuran file, file hanya menjadi satu block.
Tidak ada dua replica	Pastikan terdapat minimal dua DataNode aktif dan replication factor disetel menjadi 2.